

# Assignment 1 Report: Data Collection and Preprocessing for Foundation Model Pre-Training

**Name:** Jiao Zhang **Date:** September 27, 2026

## 1. Dataset Sources and Size

| Domain       | HF Dataset                        | Raw size                     | # Documents    |
|--------------|-----------------------------------|------------------------------|----------------|
| Encyclopedic | wikimedia/wikipedia (20231101.en) | 471.9 MB                     | 98,284         |
| General web  | Skylion007/openwebtext            | 471.9 MB                     | 93,502         |
| News         | vblagoje/cc_news                  | 209.7 MB                     | 85,943         |
| <b>Total</b> |                                   | <b>≈1,153.5 MB (1.13 GB)</b> | <b>277,729</b> |

We chose these three sources to satisfy the diversity requirement: encyclopedic (formal, structured reference prose), general web (informal, highly heterogeneous), and news (semi-formal, time-anchored, event-driven writing). Collection was done in streaming mode (`streaming=True`), so no source needed to be fully downloaded before we could start counting bytes toward its budget — each source was capped independently (472MB / 472MB / 210MB) and stopped as soon as it crossed that target, which is why the totals land slightly above each nominal target rather than exactly on it.

## 2. Cleaning Strategy and Reasoning

Steps applied, in order:

- HTML/markdown stripping** — regex removal of tags, `[text]` (`url`) link syntax, markdown headings, and reference markers (`[12]`, `[citation needed]`). These are artifacts of the source format rather than content, and would otherwise leak into the token stream as noise — Wikipedia dumps in particular carry a lot of residual markup and citation bracket noise even after HF’s own preprocessing.
- Normalization** — lowercasing and whitespace collapsing, so formatting differences between sources (extra line breaks, inconsistent casing in web text vs. news headlines) don’t inflate the vocabulary or create spurious “near-duplicates” that differ only in casing/spacing.
- Short-document filtering** (`< 50` words) and **exact-duplicate removal** (MD5 hash of normalized text), applied together across the merged corpus:

|                           | Count          | % of collected docs |
|---------------------------|----------------|---------------------|
| Total collected           | 277,729        | 100%                |
| Dropped — too short       | 13,727         | 4.9%                |
| Dropped — exact duplicate | 8,387          | 3.0%                |
| <b>Kept</b>               | <b>255,615</b> | <b>92.0%</b>        |

---

The short-document drop rate (4.9%) is consistent with what we’d expect from OpenWebText and CC-News, which both contain a fair number of stub pages, boilerplate listings, and short wire-service blurbs. The duplicate rate (3.0%) is plausible for CC-News in particular, where the same syndicated story is often crawled from multiple outlets, and for Wikipedia, where redirect/stub pairs can produce near-identical text.

**Limitation:** exact-match hashing only catches verbatim duplicates. It would miss *near*-duplicates — e.g., a wire story republished with a different byline or a lightly edited stub. A production pipeline would add MinHash/LSH-based near-deduplication on top of this; we discuss this as an extension below.

### 3. Tokenization Choices

- **Tokenizer:** GPT-2 BPE tokenizer (`AutoTokenizer.from_pretrained("gpt2")`) — vocabulary size 50,257.
- **Block size:** 1,024 tokens, matching GPT-2’s context window.
- **Strategy:** concatenate-then-chunk. Each cleaned document is tokenized and appended to a running buffer separated by an EOS token; once the buffer holds  $\geq 1,024$  tokens, a fixed-size block is sliced off and written out. This is the standard approach for GPT-style pretraining because it avoids wasting compute padding every short document up to the full block size — padding is only needed for the corpus’s final partial block.
- **Resulting corpus:** 245,713 blocks of 1,024 tokens each —  **$\approx 251.6$  million tokens** total.
- The tokenizer logged a routine warning (“sequence length ... longer than the specified maximum ... 8524 > 1024”) when encoding especially long documents (mostly long-form Wikipedia articles). This is expected and harmless here: the warning describes what would happen if a single document were fed directly to a model with no chunking, but our concatenate-then-chunk step is exactly what prevents that — the long document is simply split across several output blocks.

### 4. Data Loader Implementation

- Implemented as a PyTorch `IterableDataset` rather than a map-style `Dataset`, because at 245,713 blocks ( $\approx 1$  GB tokenized) building and holding a random-access index in memory is unnecessary overhead for a streaming read pattern.
- **Shuffling:** since iterable datasets can’t use PyTorch’s default random-access shuffle, we use a reservoir-style shuffle buffer (size 10,000 blocks): each new block triggers a random eviction from the buffer, approximating a full shuffle without loading the whole tokenized file into memory at once.
- **Batching/padding:** a custom `collate_fn` stacks blocks into a tensor and builds an attention mask. Since blocks are already fixed-length, padding logic mainly exists to safely handle the corpus’s one final partial block.
- **Streaming from disk:** the dataset reads `tokenized.jsonl` line-by-line rather than loading it fully into memory, keeping the loader’s memory footprint roughly constant regardless of corpus size.
- **Verification:** 8 sample batches were drawn and saved to `sample_dataset.pt`; each has `input_ids` shape (8, 1024), confirming correct batching and block length.

### 5. Challenges Encountered

- **Stale dataset identifiers:** the assignment’s own example dataset name, `cc_news`, no longer resolves on the Hub — HuggingFace now requires all dataset ids to be in `namespace/name` form, and `cc_news` has been migrated to `vblagoje/cc_news`. This surfaced as an `HfUriError` at collection time and was fixed by updating the config to the new id.
- **Memory:** collecting from OpenWebText/CC-News without streaming would have required downloading tens of GB before any filtering could happen; streaming mode plus stopping each source as soon as its byte budget was hit avoided this — none of the three sources were fully materialized on disk.
- **Deduplication cost:** storing one MD5 hash (16 bytes) per document in memory scales comfortably to the corpus size we used (277,729 hashes  $\approx$  4.4 MB), but a naive “store the whole normalized string in a set” approach would not scale nearly as well at larger corpus sizes.
- **Tokenization throughput:** tokenizing 255,615 documents on CPU (no GPU needed for BPE tokenization, but still single-threaded in this implementation) was the slowest stage of the pipeline — noticeably slower than collection or cleaning, since it touches every kept document individually before chunking.
- **Variable document length vs. fixed block size:** the concatenate-then-chunk strategy sidesteps padding every short document individually, at the cost of occasionally letting one training example span the boundary between two unrelated documents. The EOS separator inserted between documents gives the model a signal to treat that point as a hard context break, but it’s a trade-off worth naming rather than a fully solved problem.

## 6. Reflections on Preprocessing Impact on Pretraining Quality

Skipping deduplication in this run would have let roughly 3% of the corpus (8,387 documents) recur as exact repeats, concentrated in CC-News (where the same wire story is often crawled from several outlets) and in Wikipedia (stub/redirect pairs with near-identical bodies). At this scale that’s a mild effect, but the mechanism doesn’t stay mild as it compounds: a model trained on this data sees those repeated documents proportionally more often, which nudges it toward memorizing them rather than generalizing from them — and if a duplicate happens to also appear in whatever held-out set is used to evaluate the model, its performance there will look better than it actually is, simply because it partially memorized the eval data during training. Exact-hash dedup catches the cheap, obvious case here; the remaining risk is near-duplicates (a story re-run with a tweaked headline, a stub page that’s 90% identical to its parent article), which this pipeline doesn’t catch and which would need something like MinHash/LSH to address.

The `min_words = 50` threshold is a blunt instrument trading corpus size against document coherence. Set it lower, and more stub pages and boilerplate survive into the corpus — cheap to collect, but each one contributes a training example with almost no usable context, effectively diluting the signal-to-noise ratio of the batch it lands in. Set it higher, and the corpus shrinks (13,727 documents, 4.9% of what we collected here, were already cut at the 50-word line) while trending toward more encyclopedic or long-form content, since we’re implicitly filtering by genre as much as by quality — a short but complete recipe or a short news brief is real, coherent text that a higher threshold would discard anyway. There’s no length threshold that separates “low-quality” from “short” perfectly; 50 words is a reasonable default for general-purpose pretraining, not a principled cutoff.

Finally, domain balance directly shapes what the model learns to sound like. Here Wikipedia and OpenWebText are roughly equal in raw bytes (471.9MB each) with CC-News smaller (209.7MB) — meaning the model sees noticeably fewer news-style examples than encyclopedic or general-web examples per training epoch. If one domain dominated instead — say, CC-News alone — the model would likely pick up a more uniform, terse, headline-driven register and struggle to produce the longer, more structured explanatory prose that Wikipedia-style text teaches; the reverse (all-Wikipedia) would likely undertrain the model on casual, conversational register almost entirely absent from encyclopedic writing. The three-way split used here is a reasonable approximation of general-purpose diversity, but it’s worth noting that “diverse” here still means three fairly narrow genres, not a representative sample of language use as a whole.